

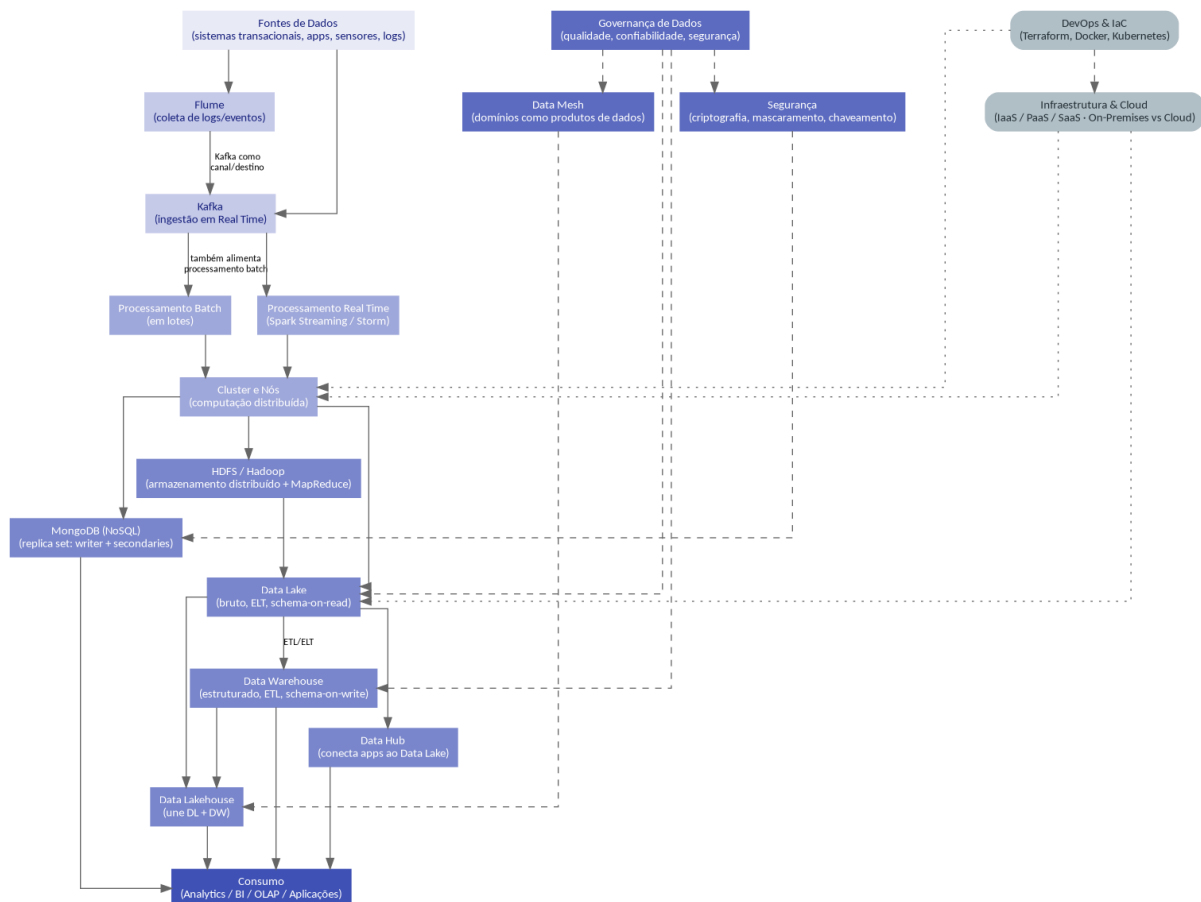
Engenharia de Dados

Resumo de Aprendizados

Este documento consolida, corrige e amplia as anotações originais de estudo sobre Engenharia de Dados, com foco no entendimento teórico dos principais conceitos da área. Também apresenta um fluxograma integrado mostrando como as subáreas (bancos de dados, processamento, governança, armazenamento, infraestrutura, DevOps, segurança e ingestão) se relacionam dentro de um pipeline de dados típico.

Visão geral integrada

O fluxo abaixo resume como os temas estudados se conectam: dados nascem em sistemas de origem, são ingeridos (em lote ou tempo real), processados em clusters distribuídos, armazenados em diferentes modelos (Data Lake, Data Warehouse, Lakehouse, NoSQL) e, por fim, consumidos por aplicações analíticas. Governança e segurança perpassam todas as camadas de armazenamento, enquanto infraestrutura, nuvem e DevOps sustentam a operação de todo o pipeline.



Linhas contínuas indicam o fluxo principal dos dados; linhas tracejadas indicam a atuação transversal da governança/segurança; linhas pontilhadas indicam o suporte de infraestrutura e DevOps a todo o ambiente.

1. Bancos de Dados Relacionais

Bancos de dados relacionais organizam a informação em tabelas normalizadas, reduzindo redundância e garantindo integridade.

1.1 OLTP x OLAP

OLTP (Online Transaction Processing) — sistemas voltados a transações rápidas, frequentes e de curta duração — ex.: sistemas de vendas, caixas eletrônicos, e-commerce.

OLAP (Online Analytical Processing) — sistemas voltados a consultas analíticas complexas sobre grandes volumes de dados históricos/agregados — ex.: dashboards de BI, relatórios gerenciais.

OLTP e OLAP não são concorrentes — eles são complementares, e normalmente representam camadas diferentes de um mesmo pipeline de dados.

1.2 Normalização

Bancos relacionais bem projetados seguem formas normais para eliminar redundância. A 3ª Forma Normal (3FN) é o padrão mais comum na prática, eliminando dependências transitivas entre colunas. Exemplos de SGBDs relacionais: MySQL, SQL Server, PostgreSQL, Oracle Database.

1.3 Linguagem SQL

- DDL (Data Definition Language) — define a estrutura dos dados: CREATE, ALTER, DROP.
- DML (Data Manipulation Language) — inclui, altera e consulta dados: INSERT, UPDATE, DELETE, SELECT.
- DCL (Data Control Language) — controla a segurança/permisões no SGBD: GRANT, REVOKE.

Conceitos básicos do modelo relacional: Entidade = Tabela; Coluna = Atributo.

1.4 Controle transacional

- BEGIN TRANSACTION — inicia um bloco de alterações.
- ROLLBACK — descarta as alterações feitas na transação.
- COMMIT — confirma e grava as alterações de forma definitiva.

Complemento: esse controle existe para garantir as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), que asseguram que uma transação seja executada por completo ou não seja executada, mesmo em caso de falhas.

2. Processamento de Dados

De modo geral, existem dois grandes paradigmas de processamento:

- Batch — processamento em lotes, com maior volume e menor urgência de tempo de resposta.
- Real Time (RT) — processamento em tempo real ou quase real, com baixa latência.

Cluster — conjunto de computadores que trabalham de forma coordenada como se fossem um único sistema.

Nó — cada computador individual que compõe um cluster.

Ferramentas citadas: Apache Storm, voltado a processamento Real Time evento a evento; e Apache Spark Streaming, que processa dados em micro-lotes (micro-batches), sendo uma abordagem híbrida entre batch e streaming puro.

3. Governança de Dados

Governança de dados é o conjunto de práticas de melhoria contínua e gerenciamento dos dados, garantindo qualidade, confiabilidade e segurança, por meio de regras e diretrizes que orientam os fluxos de dados na organização.

Governança Federada — modelo em que cada área de negócio tem autonomia para aplicar suas próprias regras dentro de diretrizes globais — de forma similar a estados de um país, que têm particularidades dentro de uma legislação nacional comum (meio termo entre centralizada e descentralizada).

Governança Centralizada — Uma equipe/área central (geralmente ligada a TI ou a um Chief Data Officer) define e aplica todas as regras, padrões e políticas para toda a organização.

Governança Descentralizada — Cada área de negócio (marketing, vendas, financeiro etc.) cuida totalmente da governança dos seus próprios dados, sem um órgão central coordenando.

Direcionamento Data Driven — abordagem em que o dado é usado como principal direcionador das tomadas de decisão, em vez de intuição ou experiência isolada.

Data Mesh — arquitetura de dados descentralizada que organiza os dados por domínio de negócio, tratando cada domínio como responsável por seus próprios "produtos de dados".

Complemento: Data Mesh pode ser entendido como uma evolução/implementação prática dos princípios da governança federada: ele aplica a ideia de autonomia por domínio, mas vai além, definindo ownership claro dos dados, infraestrutura self-service e o conceito de "dado como produto".

4. Modelos de Armazenamento Analítico

Data Warehouse (DW) — repositório de metadados centralizados, com dados estruturados (SQL), alimentado por processos de ETL (Extract, Transform, Load) — os dados são transformados antes de serem carregados (schema-on-write).

Data Lake — repositório de grande volume de dados em diferentes formatos (estruturados e não estruturados, SQL e NoSQL), alimentado por processos de ELT (Extract, Load, Transform) — os dados são carregados brutos e transformados apenas no momento do uso (schema-on-read).

Complemento: o Data Lake costuma ser mais barato que o Data Warehouse porque utiliza armazenamento de baixo custo (ex.: object storage/commodity hardware) e não exige transformação prévia dos dados; o Data Warehouse, por outro lado, envolve processamento e estruturação antecipados, além de mecanismos de consulta otimizados, o que eleva o custo de armazenamento e computação.

Data Lakehouse — arquitetura que une a flexibilidade e o baixo custo do Data Lake com a confiabilidade transacional (ACID) e a performance de consultas do Data Warehouse, geralmente via camadas como Delta Lake, Apache Iceberg ou Apache Hudi.

Complemento: essa é a diferença entre Data Lake e Data Lakehouse: o Lake armazena dados brutos sem muitas garantias; o Lakehouse adiciona controle transacional, versionamento e schema enforcement sobre o próprio Lake, permitindo BI direto sobre ele.

Data Hub — camada que conecta as aplicações ("Apps") de cada área de negócio a um Data Lake central, facilitando o compartilhamento de dados entre sistemas.

"Data Pond" — termo informal para um subconjunto menor e mais específico de um Data Lake, geralmente dedicado a um time ou projeto — não deve ser confundido com o sistema de arquivos distribuído do Hadoop (HDFS).

5. Infraestrutura e Computação em Nuvem

IaaS (Infrastructure as a Service) — o provedor entrega apenas a infraestrutura (servidores, rede, armazenamento); o cliente gerencia sistema operacional, runtime e aplicações.

PaaS (Platform as a Service) — o provedor entrega a infraestrutura e a plataforma/sistema de execução; o cliente foca apenas em desenvolver e manter a aplicação.

SaaS (Software as a Service) — o provedor entrega tudo, incluindo o próprio software pronto para uso pelo cliente final.

5.1 On-Premises x Cloud

- On-Premises: infraestrutura própria, mantida no data center da empresa; modelo de investimento CAPEX (alto investimento inicial); controle total sobre hardware e dados; indicado quando há exigências rígidas de compliance, latência ou customização de hardware.
- Cloud: infraestrutura de terceiros, consumida sob demanda; modelo de investimento OPEX (pagamento pelo uso); alta elasticidade e escalabilidade; indicado para cargas de trabalho variáveis, times enxutos de infraestrutura e necessidade de rapidez para escalar.
- Modelos comuns de cobrança em nuvem: pay-as-you-go (paga-se pelo uso efetivo), instâncias reservadas (desconto por compromisso de longo prazo) e instâncias spot (capacidade ociosa mais barata, porém sujeita a interrupção).

5.2 Instância x Cluster

Instância — uma máquina virtual (VM) individual.

Cluster — um conjunto de instâncias/nós que trabalham de forma coordenada, como se fossem um único sistema, geralmente para escalabilidade e tolerância a falhas.

6. DevOps, Infraestrutura como Código e Contêineres

DevOps é a tendência de integração entre equipes de desenvolvimento e operação, gerando um fluxo de entrega mais rápido, maior qualidade e maior tolerância a falhas. A Infraestrutura como Código (IaC) é uma grande aliada do DevOps, principalmente para automação de testes e provisionamento consistente de ambientes.

6.1 Shift Left e Shift Right

- Shift Left — mover testes, segurança e validações de qualidade para etapas mais iniciais do ciclo de desenvolvimento (o foco vai da operação para o código, antecipando problemas).
- Shift Right — validar e monitorar o comportamento do sistema já em produção (o foco vai do código para a operação), incluindo testes A/B, observabilidade e monitoramento contínuo.

6.2 Infraestrutura como Código (IaC)

IaC significa descrever e provisionar infraestrutura (servidores, redes, bancos de dados) por meio de arquivos de configuração versionáveis, em vez de configuração manual. O Terraform é um exemplo de ferramenta declarativa de IaC: o usuário descreve o estado desejado da infraestrutura, e a ferramenta se encarrega de criar, atualizar ou remover os recursos necessários na nuvem para atingir esse estado.

6.3 Docker e Kubernetes

O Docker permite empacotar uma aplicação e suas dependências em uma imagem — um pacote com todas as configurações necessárias para a aplicação rodar. Essa imagem é então executada dentro de um contêiner (uma instância isolada da imagem), o que permite rodar testes e aplicações de forma padronizada em qualquer ambiente.

O Kubernetes é uma ferramenta de orquestração de contêineres: ele gerencia automaticamente a criação, o escalonamento, a autorrecuperação (self-healing) e o balanceamento de carga entre múltiplos contêineres, de forma similar a gerenciar uma equipe de atividades em execução simultânea.

6.4 Integração entre Terraform e Docker

Terraform e Docker atuam em camadas diferentes, e complementares, de um mesmo pipeline de automação:

- O Docker resolve o "o quê" roda: empacota a aplicação e suas dependências em uma imagem, garantindo que ela se comporte da mesma forma em qualquer ambiente.
- O Terraform resolve o "onde" e "em cima de quê" isso roda: provisiona a infraestrutura (rede, servidores, cluster) necessária para executar esses contêineres.

Fluxo típico de integração entre as duas ferramentas:

- 1. A imagem Docker da aplicação é construída (build) e publicada em um registro de imagens (ex.: Docker Hub, Amazon ECR, Google Artifact Registry).
- 2. O Terraform é executado para provisionar a infraestrutura de destino — por exemplo, uma instância com Docker instalado, um cluster Kubernetes (EKS/GKE/AKS) ou um serviço gerenciado de contêineres (ex.: AWS ECS/Fargate).
- 3. Dentro do próprio código do Terraform, o recurso de infraestrutura (a instância, o serviço ECS, o deployment do Kubernetes) é configurado para referenciar a imagem Docker publicada no passo 1, informando qual imagem deve ser baixada e executada.
- 4. Ao aplicar o plano ("terraform apply"), a infraestrutura é criada (ou atualizada) já pronta para puxar a imagem do registro e subir os contêineres correspondentes.

Complemento: na prática, o Terraform normalmente não constrói a imagem Docker (isso fica a cargo do pipeline de CI/CD, ex.: GitHub Actions/GitLab CI); o papel do Terraform é garantir que a infraestrutura onde essa imagem vai rodar exista, esteja configurada corretamente (rede, permissões, variáveis de ambiente) e aponte para a versão certa da imagem — unindo IaC (infraestrutura) e containers (aplicação) em um único fluxo automatizado.

7. Segurança e Criptografia de Dados

A segurança de dados envolve diversas técnicas complementares:

- Criptografia em repouso — protege dados armazenados (em disco/banco); criptografia em trânsito — protege dados durante a transmissão pela rede.
- Criptografia simétrica — mesma chave para cifrar e decifrar (mais rápida); criptografia assimétrica — par de chaves pública/privada (mais segura para troca de chaves).
- Mascaramento de dados — substitui dados sensíveis por valores fictícios, mantendo o formato original; muito usado em ambientes de teste e desenvolvimento.
- Tokenização — substitui um dado sensível por um token sem valor explorável, mantendo a referência ao dado original em um cofre seguro.

- Hashing — transforma um dado em um valor de tamanho fixo, de forma unidirecional (não reversível), usado por exemplo para senhas.
- Chaveamento / gerenciamento de chaves (key management) — controla a geração, distribuição, rotação e revogação das chaves criptográficas.

8. Big Data e Armazenamento Distribuído

8.1 Teorema CAP

Em um sistema distribuído, nunca é possível garantir simultaneamente as três propriedades abaixo — apenas duas delas ao mesmo tempo:

- Consistência — todos os nós enxergam o mesmo dado ao mesmo tempo.
- Disponibilidade — o sistema sempre responde a uma requisição, mesmo que algum nó esteja indisponível.
- Tolerância a Partições — o sistema continua funcionando mesmo quando há falha de comunicação entre os nós.

8.2 Hadoop e HDFS

O Hadoop é um sistema de arquivos distribuído (HDFS — Hadoop Distributed File System): ele divide grandes arquivos em blocos e os distribui entre diversos nós de um cluster.

Complemento: o HDFS garante maior consistência e tolerância a falhas porque replica cada bloco de dado em múltiplos nós (fator de replicação padrão igual a 3); assim, se um nó falhar, o dado continua disponível em outros nós, sem perda de informação.

8.3 MapReduce

MapReduce é um paradigma de processamento paralelo e distribuído, dividido em duas fases principais:

- Map — os dados de entrada são divididos em partes e processados em paralelo por diferentes nós, gerando pares chave-valor intermediários.
- Reduce — os resultados intermediários são agrupados por chave e agregados/combinaos, gerando o resultado final.

Fluxo resumido: Dados de entrada → Divisão (Split) → Map (processamento paralelo) → Shuffle/Sort (agrupamento por chave) → Reduce (agregação) → Saída.

9. Bancos NoSQL — MongoDB

O MongoDB é um banco de dados NoSQL orientado a documentos. Em uma configuração de alta disponibilidade (Replica Set), o driver do cliente envia as escritas para um nó primário (writer) e as leituras podem ser distribuídas entre nós secundários (readers), que mantêm cópias replicadas dos dados.

Failover — processo pelo qual o sistema troca automaticamente para outro nó (por exemplo, elegendo um novo primário entre os secundários) quando o nó principal fica indisponível, garantindo continuidade do serviço.

10. Armazenamento em Cache e Estratégias de Busca

O cache é um armazenamento intermediário, normalmente em memória RAM, usado para acelerar o acesso a dados frequentemente utilizados.

Complemento: o cache em RAM é mais rápido que o armazenamento tradicional porque evita o acesso a camadas mais profundas de memória (como disco), que possuem maior latência; ao manter os dados mais acessados próximos do processamento, reduz-se o tempo de resposta.

Estratégias comuns de busca em sistemas de armazenamento:

- Busca sequencial — percorre todos os registros até encontrar o desejado; simples, porém custosa em grandes volumes.
- Busca indexada — utiliza estruturas como B-Tree ou Hash Index para localizar rapidamente um registro sem percorrer todos os dados.
- Busca full-text — indexação voltada a texto livre/dados não estruturados, permitindo buscas por palavras-chave e relevância.

11. Ingestão de Dados

Kafka — plataforma de mensageria distribuída (publish/subscribe), voltada à ingestão de dados em tempo real, organizada em tópicos e partições.

Spark — motor de processamento distribuído, tradicionalmente associado a processamento batch, mas que também processa streaming (via Spark Streaming/Structured Streaming, em micro-lotes).

Exemplo de uso conjunto: o Kafka pode capturar em tempo real os eventos de cliques em um site; o Spark Structured Streaming consome esses tópicos para calcular métricas quase em tempo real (ex.: cliques por minuto), enquanto o Spark em modo batch processa, periodicamente, análises históricas mais pesadas sobre o mesmo conjunto de dados já armazenado.

Flume — ferramenta voltada à coleta de logs e eventos, com destino tipicamente ao HDFS.

Kafka e Flume podem ser usados de forma integrada: o Flume pode utilizar o Kafka como canal (Kafka Channel) entre a fonte de dados e o destino final, aproveitando a durabilidade e a capacidade de reprocessamento (replay) do Kafka, junto com a simplicidade de configuração do Flume para mover dados até o Data Lake.

12. Ferramentas de ETL/ELT — Pentaho Data Integration (PDI)

O PDI (Pentaho Data Integration) é uma ferramenta open source de ETL/ELT com interface visual, cujo ambiente gráfico de construção de fluxos é chamado Spoon.

Quando usar: o PDI é indicado para prototipagem rápida de pipelines, integração entre múltiplas fontes de dados sem a necessidade de codificação extensa, e em cenários onde parte da equipe não é totalmente técnica ou onde há necessidade de entregar integrações rapidamente.

13. Modelo DIKW

O modelo DIKW descreve a evolução do dado bruto até a sabedoria, em quatro camadas:

- Data (Dado) — fato bruto, sem contexto (ex.: o número 37).
- Information (Informação) — dado contextualizado, com significado (ex.: temperatura de 37°C).
- Knowledge (Conhecimento) — informação combinada com experiência/padrões, permitindo interpretação (ex.: 37°C indica febre em uma pessoa).

- Wisdom (Sabedoria) — aplicação do conhecimento com julgamento, sabendo quando e como agir (ex.: decidir procurar atendimento médico).

A Engenharia de Dados atua principalmente nas camadas de Dado e Informação (coleta, ingestão, armazenamento, organização), e cada vez mais alcança a camada de Conhecimento por meio de analytics avançado e machine learning — enquanto a Sabedoria permanece uma camada predominantemente humana e estratégica.

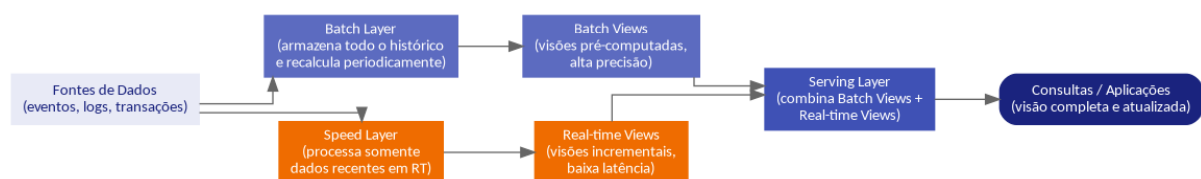
14. Arquitetura Lambda

A Arquitetura Lambda é um padrão de arquitetura de dados criado para resolver um problema comum nos pipelines vistos até aqui: como ter, ao mesmo tempo, precisão sobre todo o histórico de dados (típico do processamento Batch) e baixa latência sobre os dados mais recentes (típico do processamento Real Time), sem sacrificar um em favor do outro.

14.1 Objetivo

O objetivo é oferecer uma visão de dados completa e sempre atualizada, combinando dois caminhos de processamento que rodam em paralelo sobre a mesma fonte de dados — um focado em exatidão sobre grandes volumes históricos, outro focado em velocidade sobre os eventos mais recentes — e depois unificar os resultados dos dois em uma única camada de consulta.

14.2 Fluxo



A arquitetura é dividida em três camadas:

Batch Layer — recebe todos os dados brutos (geralmente vindos de um Data Lake) e recalcula periodicamente visões completas e precisas sobre todo o histórico — as Batch Views. É lenta, mas exata, e serve como "fonte da verdade".

Speed Layer — processa em tempo real apenas os dados mais recentes, que ainda não foram incorporados pela última execução da Batch Layer, gerando Real-time Views. É rápida, porém aproximada/incremental, cobrindo a "janela" entre uma execução batch e outra.

Serving Layer — combina as Batch Views (precisas, mas com atraso) com as Real-time Views (recentes, mas aproximadas) para responder às consultas com uma visão unificada e atualizada dos dados.

14.3 Onde entra no fluxo de Engenharia de Dados

A Arquitetura Lambda não substitui os conceitos vistos nas seções anteriores — ela organiza e reaproveita várias dessas peças dentro de um único padrão:

- As Fontes de Dados e a Ingestão (Kafka, Flume) alimentam simultaneamente a Batch Layer e a Speed Layer.
- A Batch Layer normalmente se apoia em Data Lake/HDFS para armazenamento e em motores de processamento batch (Spark, MapReduce) para recalculas as visões.
- A Speed Layer se apoia em ferramentas de processamento em Real Time (Spark Streaming, Storm) para gerar resultados incrementais de baixa latência.

- A Serving Layer costuma expor os resultados em um Data Warehouse, Data Lakehouse ou banco NoSQL de leitura rápida, de onde as aplicações e ferramentas de Analytics/BI consultam os dados.

***Complemento:** um ponto de atenção da Arquitetura Lambda é a necessidade de manter e evoluir duas bases de código (uma para a lógica batch, outra para a lógica real-time), o que aumenta a complexidade de manutenção — esse é justamente o problema que a Arquitetura Kappa (uma variação que usa apenas uma camada de streaming para tudo) tenta resolver.*

Considerações finais

Os temas acima não são isolados: como mostra o fluxograma da seção inicial, uma arquitetura real de dados combina ingestão (Kafka/Flume), processamento distribuído (clusters, Spark, MapReduce), armazenamento em múltiplos modelos (Data Lake, Data Warehouse, Lakehouse, NoSQL) e camadas transversais de governança, segurança, infraestrutura e DevOps. Entender essas conexões é o que permite migrar de conceitos isolados para uma visão de arquitetura de dados de ponta a ponta.

Glossário

Batches – lotes de dados processados de uma só vez (processamento em lote), em oposição ao processamento contínuo em tempo real.

Lambda – termo ambíguo, com dois usos comuns: (1) a Arquitetura Lambda, que você já tem no documento; (2) AWS Lambda, um serviço de computação serverless que executa código em resposta a eventos, sem que você precise provisionar ou gerenciar servidores.

Queries ad-hoc – consultas pontuais, criadas sob demanda para responder a uma pergunta específica, sem estarem previamente programadas/agendadas em um relatório fixo.

Hardware – os componentes físicos de um sistema computacional (processador, memória, disco, placa de rede etc.), em oposição ao software.

Software – conjunto de programas, instruções e dados que dizem ao hardware o que fazer; é a parte lógica de um sistema computacional (sistema operacional, aplicativos, scripts etc.), em oposição ao hardware, que é a parte física.

Firewall – sistema de segurança (físico ou lógico) que filtra o tráfego de rede, permitindo ou bloqueando conexões com base em regras predefinidas.

Latência – tempo de atraso entre o envio de uma solicitação e o recebimento da resposta.

I/O – Input/Output (entrada/saída): operações de leitura e escrita de dados entre o processador/memória e dispositivos externos (disco, rede, etc.).

DBA – Database Administrator: profissional responsável por administrar, otimizar e proteger bancos de dados.

SWAP – área do disco usada como extensão da memória RAM quando ela se esgota, permitindo que o sistema continue funcionando (com desempenho reduzido).

Datacenter – instalação física dedicada a hospedar servidores, redes e demais equipamentos de infraestrutura de TI.

Dbeaver – ferramenta gráfica (cliente SQL universal) usada para conectar, consultar e administrar diferentes bancos de dados por meio de uma interface visual.

Endpoint – ponto de acesso de uma API ou serviço (geralmente uma URL) através do qual sistemas trocam dados entre si. (de onde você puxa um dado de um sistema de API – por exemplo, de onde eu puxo os dados do SGF ou do SAP, o ‘endereço’ de onde eu puxo o banco através da API).

Staging area – área intermediária de um pipeline ETL onde os dados brutos ficam temporariamente armazenados após a extração, antes de serem transformados e carregados no destino final. (Fonte de Dados → [STAGING AREA] → Transformação → Data Warehouse / Data Lake)

Servidores de cache – servidores dedicados a armazenar dados temporários em memória para acelerar o acesso a informações usadas com frequência (ex.: Redis, Memcached).

Sistema de CRM – Customer Relationship Management: sistema de gestão do relacionamento com o cliente, usado para centralizar dados de vendas, atendimento e marketing.

Server Message Block – protocolo de rede usado para compartilhamento de arquivos, impressoras e recursos entre sistemas, comum em ambientes Windows.

Redundância de disponibilidade – duplicação de componentes, servidores ou dados para garantir que o sistema continue disponível mesmo se uma parte falhar (base da alta disponibilidade/high availability).

CI/CD – Continuous Integration / Continuous Delivery (ou Deployment): práticas de DevOps que automatizam a integração, os testes e a entrega (ou implantação) contínua de código em produção.